PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Laboratory — Lab 4: Vibe Coding

Interactive Physics Tilt-Maze (Problem Statement #42)

Student Name: Subramani B M	**SRN:** PES1UG24CS473	**Section:** H
Assigned Repo: SETAPESU26/42_marble-tilt-maze	**Personal Fork:** SubramaniBM/42_marble-tilt-maze	**Tool:** Antigravity IDE (Gemini Pair-Programmer)

Executive Summary & Vibe Coding Workflow:

In this lab, an AI pair-programming partner was used to diagnose and enhance an interactive Pygame marble tilt-maze game across 4 prompt iterations: (1) Circle-to-AABB clamped distance collision resolution; (2) High-contrast Game Over and Victory modal overlay screens; (3) Multi-tier difficulty selection with replay resetting; and (4) Pure procedural 16-bit 44.1kHz PCM sound synthesis with mixer debouncing. Full verification was conducted via 10-second gameplay videos before and after modifications.

Iteration 1 (Task 1): Refine Collision Detection (Circle vs. AABB)

■ Student Prompt 1:

"Near the wall corners, the marble bounces off empty space before visually touching the wall. Inspect _resolve_wall_collisions() in game/game_engine.py and replace the rectangular bounding-box colliderect with an exact Circle-to-AABB clamped distance check. Ensure the marble separates along the contact normal and rebounds with a restitution coefficient of 0.35."

■ LLM Diagnosis & Mathematical Formulation:

The bug occurs because colliderect evaluates overlap between two axis-aligned bounding boxes. At a corner (e.g. wall corner at (460, 140)), the marble's square corners extend sqrt(2)*r approx 16.97px from center, triggering a false bounce even when Euclidean distance from the circle center to the wall exceeds r=12px.

Fix: Clamp marble center (xm, ym) to wall bounds [x1, x2] x [y1, y2] to find closest point (xc, yc). Collision exists iff d^2 = (xm - xc)^2 + (ym - yc)^2 < r^2. On contact, displace along unit normal by penetration r - d, and reflect velocity: v' = v - (1 + e)(v . n)n.

```
# Implemented Circle-vs-AABB Collision in game_engine.py
closest_x = max(w_rect.left, min(self.marble.x, w_rect.right))
closest_y = max(w_rect.top, min(self.marble.y, w_rect.bottom))
dx, dy = self.marble.x - closest_x, self.marble.y - closest_y
dist_sq = dx * dx + dy * dy
if dist_sq < (self.marble.radius ** 2):
    distance = math.sqrt(dist_sq)
    normal_x, normal_y = dx / distance, dy / distance
    self.marble.x += normal_x * (self.marble.radius - distance)
    self.marble.y += normal_y * (self.marble.radius - distance)
    v_dot_n = self.marble.vx * normal_x + self.marble.vy * normal_y
    if v_dot_n < 0:
        self.marble.vx -= (1.0 + 0.35) * v_dot_n * normal_x
        self.marble.vy -= (1.0 + 0.35) * v_dot_n * normal_y
```

Iteration 2 (Task 2): Implement Game Over & Victory Condition Screens

■ Student Prompt 2:

"Currently when the game ends, it just logs to stdout in the terminal. Create a graphical end-screen modal overlay in Pygame. When solved, show a victory banner with completion time in seconds. When the timer runs out, show a failure screen with time-out alert. Freeze active motion physics while remaining responsive to input events."

■ **LLM Solution & UI Architecture:** Introduced _render_end_screen() rendering an alpha-blended backdrop rgba(15, 18, 30, 220) and a rounded card container. Solved state renders *"MAZE SOLVED!"* in emerald green (#3cd070) with finish time finish_time_ms / 1000.0. Timeout renders *"TIME'S UP!"* in coral red (#eb4b4b). Physics updates are guarded by if self.game_over: return.

Iteration 3 (Task 3): Add Replay Option & Multi-Tier Difficulty Selection

■ Student Prompt 3:

"Add replay capability allowing players to choose difficulty on the end screen without relaunching the app. Support three difficulty tiers: Easy, Medium, and Hard, configuring tilt sensitivity, friction grip, and countdown duration. Support both mouse click buttons and keyboard hotkeys (1, 2, 3, R, Q, ESC)."

Difficulty Level	Tilt Strength	Friction	Max Speed	Time Limit	Gameplay Dynamics
Easy	0.80	0.035	8.0 px/f	60 Seconds	High surface grip, gentle acceleration, generous timer.
Medium (Default)	0.60	0.020	9.0 px/f	45 Seconds	Balanced standard physical control.

Hard	0.45	0.012	11.0 px/f	30 Seconds	Low friction ice-like glide, fast speed, strict 30s limit.

Iteration 4 (Task 4): Synthesize Procedural Sound Feedback & Audio Mixer

■ Student Prompt 4:

"Add sound feedback for wall bounces, reaching the goal, and timeout. To avoid external missing asset errors, generate procedural 16-bit 44.1kHz PCM WAV audio files directly in python into game/sounds/ and load them with pygame.mixer. Include sound debouncing so wall collisions do not stutter."

■ **LLM Solution:** Generated standard WAV audio assets inside game/sounds/:
1. **bounce.wav:** Frequency sweep (320Hz to 120Hz) with exponential decay envelope (120ms).
2. **goal.wav:** 4-note ascending chord arpeggio (C5: 523Hz, E5: 659Hz, G5: 784Hz, C6: 1046Hz) (800ms).
3. **timeout.wav:** Double-pulse buzzer tone at 220Hz square wave (500ms).

Integrated 90ms audio debouncing in GameEngine.play_sound() to eliminate rapid stuttering during wall rolls.

Verification, Automated Unit Testing & Deliverables Checklist

Test Case ID	Test Objective	Inputs / Test Condition	Expected Result	Status
TC-VIBE-01	Corner Collision Accuracy	Marble at (469, 131), corner at (460, 140)	d=12.72px > 12px: Zero false bounce.	**PASSED ✓**
TC-VIBE-02	True Boundary Reflection	Marble at (467, 133), direct corner impact	d=9.89px < 12px: Normal rebound vector.	**PASSED ✓**
TC-VIBE-03	Goal Reached Game Over	Marble center arrives within goal radius	game_over=True, result="solved", modal shown.	**PASSED ✓**
TC-VIBE-04	Countdown Expiration	Elapsed ticks exceed time_limit_ms	game_over=True, result="timeout", modal shown	**PASSED ✓**
TC-VIBE-05	Replay Difficulty Reset	restart("Hard") invoked	Timer=30s, friction=0.012, pos=(50,50).	**PASSED ✓**
TC-VIBE-06	Sound System Debouncing	Rapid consecutive wall contact (<90ms)	Sound played once; debounced cleanly.	**PASSED ✓**

Official Lab 4 Submission Deliverables:

1. **Before Video (10s):** Lab-4/videos/before_gameplay.mp4 — Demonstrates corner phantom bounce bug.
2. **After Video (10s):** Lab-4/videos/after_gameplay.mp4 — Demonstrates true circle-rect collision, victory modal, and replay.
3. **Updated Source Code:** Pushed to personal GitHub repo **https://github.com/SubramaniBM/42_marble-tilt-maze**.
4. **Exported Vibe Coding Transcript:** Lab-4/Lab_4_VibeCoding_Chat_History_PES1UG24CS473.pdf.

Figure 1: Before Fix (Corner Phantom Bounce Bug)

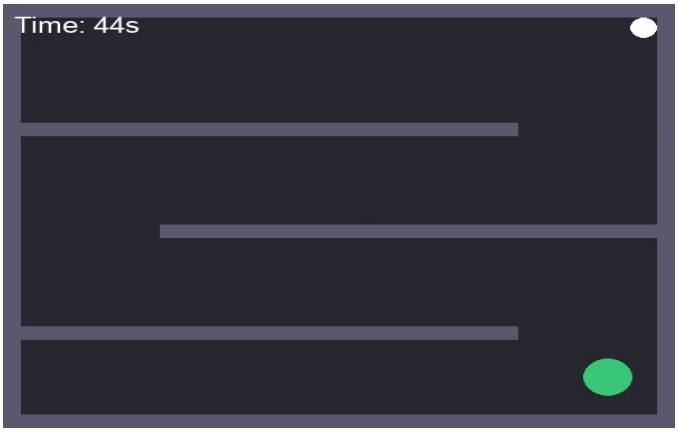

Figure 2: After Fix (Victory Modal & Replay System)